

Webshop-Architektur & Infrastruktur

1. Einführung und Projektrahmenbedingungen

In diesem Dokument werden alle relevanten Technologien, Programme und strukturellen Zusammenhänge veranschaulicht.

Unser Ziel ist es eine skalierbare und isolierte Drei-Schichten-Architektur aufzubauen, welche in einem Docker-Container-Setup lauffähig ist und über GitHub versioniert wird. Zunächst einmal...

...Was ist eine Drei-Schichten-Architektur?

Wie wir gelernt haben, ist dies eigentlich das Fundament von unserer Anwendung. Denn anstatt, dass wir alles in einen großen Programmcode zusammenpacken, wird unser Webshop in drei strikt getrennte Schichten aufgeteilt:

1. **Schicht: Präsentationsschicht (Frontend):** Das, was der Nutzer im Browser sieht. Hier laufen HTML, CSS und JavaScript.
2. **Schicht: Geschäftslogik (Backend):** Das "Gehirn" der App. Hier läuft Node.js und prüft z.B., ob ein Passwort stimmt oder ob ein Produkt noch auf Lager ist.
3. **Schicht: Datenhaltungsschicht (Datenbank):** Das "Gedächtnis". Hier speichert PostgreSQL dauerhaft (persistent) alle User, Produkte und Wunschlisten ab.

...Was versteht man unter skalierbar?

Mit skalierbar ist einfach gemeint, dass unser Webshop theoretisch erweiterbar wäre, wenn man von einem realen Kundenzuwachs ausgehen würde. Da unser Backend von unserem Frontend getrennt ist, könnten wir somit theoretisch viele weitere Node.js-Server zuschalten, um die Last abzufangen. Dafür müssen wir nichts an der Datenbank oder am Frontend ändern. Somit ist die Skalierbarkeit automatisch gegeben, wenn wir uns strikt an die Drei-Schichten-Architektur halten.

...Was versteht man unter isoliert?

Unter isoliert verstehen wir das Frontend weiß nicht, wie die Datenbank funktioniert. Das Backend weiß nicht, wie das CSS aussieht. Das ist wichtig für uns, denn wenn etwas am Design kaputtgeht, reißt es nicht direkt die sichere Datenbank mit in den Abgrund. Somit hat jede Schicht nur eine bestimmte Aufgabe und wir können damit auch schneller erkennen, wo eventuell ein Fehler liegt.

...Was versteht man unter in einem Docker-Container-Setup lauffähig?

Gerade wenn verschiedene Betriebssysteme verwendet werden, besteht oft das Problem, dass etwas auf dem einen Betriebssystem eventuell gut funktioniert und auf dem anderen dann aber nicht. Damit man gewährleisten kann, dass die Webseite auf jedem Laptop funktioniert, benötigen wir somit Docker. Dieser löst dieses Problem in dem er zum Beispiel Backend und die Datenbank jeweils in einen Container steckt. In diesem Container ist jeweils alles enthalten, was das Programm zum Laufen benötigt. Das Docker-Setup ist dabei nun eine Verknüpfung zwischen den verschiedenen Containern. So kann dann nun alles gestartet werden und die Webseite läuft auf jedem Betriebssystem exakt identisch.

→ wie wir auf GitHub arbeiten, hat Bekir bereits im Ordner Planung hochgeladen, da sollte dann eigentlich bereits alles verständlich sein.

2. Detaillierte Technologie-Übersicht

2.1 Frontend (Präsentationsschicht)

Technologien: HTML5, CSS3, JavaScript

Zweck: Diese Technologien bilden die Benutzeroberfläche im Browser. Gemäß den Anforderungen müssen wir **zwei separate Frontends** entwickeln:

1. für reguläre Kunden
2. für Administratoren.

HTML strukturiert den Inhalt. **CSS** übernimmt das Styling. **Wichtig** beim Styling ist vor allem das Responsive Design, da das 50 % der Note im Bewertungsbereich "Erscheinungsbild" ausmacht.

JavaScript implementiert die Dynamik und Validierung der Benutzereingaben sowie die asynchrone Kommunikation mit dem Backend. Dynamik ist glaube ich recht verständlich **ohne JS** müsste beispielsweise beim Hinzufügen eines Produktes in den Warenkorb die Seite jedes Mal neu geladen werden, damit sich die Anzahl der Waren im Warenkorb verändert. **Mit JS** funktioniert dies dynamisch direkt durch das Klicken. Mit Validierung ist gemeint, dass JS die eingegebenen Daten zum Beispiel bei der Registrierung sofort prüft und einen Hinweis gibt, falls etwas falsch eingegeben wurde. Dies passiert bevor die Daten an den Server gesendet werden.

Verwendete Programme / Tools: Für die Entwicklung wird primär die im Browser integrierte Entwicklerkonsole (z.B. Firefox, Edge) genutzt.

2.2 Backend (Geschäftslogik-Schicht)

Technologie: Node.js

Zweck: Node.js dient als serverseitige Laufzeitumgebung. Hier werden die fachlichen Webservices nach dem **RESTful API-Architekturmuster** implementiert.

Eine API ist eine Schnittstelle, über die das Frontend (der Browser) mit dem Backend sprechen kann, ohne zu wissen, wie das Backend intern funktioniert.

„RESTful“ sind quasi die Spielregeln an die wir uns halten müssen. Wichtig sind dabei z.B.:

- saubere URLs (z.B. /api/produkte/123)
- HTTP-Methoden wie ein Vokabular:
 - GET: Hol mir Daten (z.B. zeige alle Produkte).
 - POST: Erstelle etwas Neues (z.B. leg einen neuen User an).
 - PUT: Aktualisiere etwas (z.B. ändere den Preis eines Produkts).
 - DELETE: Lösche etwas (z.B. lösche ein Produkt aus der Wunschliste)

Die Services umfassen das Inventarsystem, die Wunschlisten-Verwaltung, Authentifizierung, Autorisierung und die User-Verwaltung. Das Backend prüft Berechtigungen, bearbeitet die Daten und sendet Antworten (meist als JSON) zurück. Zudem übernimmt das Backend den geforderten E-Mail-Versand für Kaufbestätigungen und Registrierungen.

Verwendete Programme: Als Paketmanager für Bibliotheken wird npm genutzt. Wenn z.B. E-Mails verschickt werden sollen, können hiermit bereits fertige Bibliotheken in unser Projekt geladen werden. Mit dem Programm Postman könnt wir manuell einen "GET" oder "POST" Request an unser Node.js-Backend schicken, um zu testen, ob eure RESTful API richtig funktioniert, bevor das Frontend existiert.

2.3 Datenbank (Datenhaltungsschicht)

Technologie: PostgreSQL

Zweck: Wie oben schon erwähnt, ist das unser "Gedächtnis". PostgreSQL ist das relationale Datenbankmanagementsystem, in dem alle Daten dauerhaft (persistent) gesichert werden. Aber warum brauchen wir das überhaupt? Wie wir gelernt haben, ist das HTTP-Protokoll von Natur aus zustandslos, damit ist gemeint, der Server vergisst nach einer Anfrage sofort wieder alles. Wenn ein Kunde sich also registriert oder etwas auf die Wunschliste setzt, müssen diese zustandsbehafteten Daten irgendwo sicher auf der Festplatte landen. Genau das macht PostgreSQL.

In einer relationalen Datenbank werden die Daten in Tabellenstrukturen gespeichert. Dabei nutzen wir Konzepte wie Primär- und Fremdschlüssel, um Tabellen logisch miteinander zu verknüpfen (so weiß die Datenbank z.B.: Wunschliste X gehört exakt zu Kunde Y). Zusätzlich nutzen wir Constraints (feste Regeln, wie z.B. "eine E-Mail-Adresse darf es nur einmal geben") und die sogenannten ACID-Eigenschaften. ACID ist quasi unser Sicherheitsnetz: Es garantiert, dass eine Transaktion (wie ein Kaufvorgang) immer komplett oder gar nicht ausgeführt wird. Selbst wenn der Server mittendrin abstürzt, haben wir so niemals "halbe" Käufe in der Datenbank.

Verwendete Programme: Innerhalb von Node.js wird die Bibliothek "pg" verwendet (Verbindungspooling). Es ist strikt darauf zu achten, dass

Benutzereingaben über **parametrisierte Queries** (z.B. \$1) eingefügt werden, um SQL-Injections zu verhindern.

2.4 Laufzeitumgebung und Orchestrierung

Technologie: Docker & Docker Compose

Zweck: Wie wir in der Einführung schon geklärt haben, verpackt Docker unser Projekt in isolierte Container (wir haben z.B. einen eigenen Backend-Container und einen Datenbank-Container). Dadurch läuft die Anwendung am Ende auf dem Mac des einen Teammitglieds exakt gleich wie auf dem Windows-Rechner des anderen.

Docker Compose ist dabei quasi unser Dirigent. Es regelt das Zusammenspiel der Container, stellt ein virtuelles Netzwerk bereit, damit sie miteinander reden können, und legt fest, wer zuerst starten darf.

Ganz wichtig: Container sind eigentlich flüchtig. Wenn wir einen Container neustarten, wäre unsere Datenbank normalerweise komplett leer! Damit diese mühsam gespeicherten Daten eben nicht verloren gehen, müssen wir sogenannte **Docker Volumes** in unserer Konfiguration einbinden. Diese speichern die Datenbankdaten sicher außerhalb des flüchtigen Containers ab.

Verwendete Programme / Tools: Wir arbeiten hier mit dem Docker CLI (der Kommandozeile), einem Dockerfile (welches das Aussehen des Containers definiert) und der docker-compose.yml (welche das Zusammenspiel regelt).

3. Wie die Komponenten zusammenarbeiten

Damit man sich das Ganze in der Praxis besser vorstellen kann, schauen wir uns mal an, was genau im Hintergrund passiert, wenn ein Anwender auf "Produkt in Wunschliste speichern" klickt:

1. Nutzeraktion im Browser (Frontend): Unser JavaScript fängt den Klick des Kunden direkt ab. Anstatt die Seite neu zu laden, sendet es lautlos im Hintergrund (asynchron per fetch) einen HTTP POST-Request an unser Backend. Die Infos zum Produkt werden dabei als kleines JSON-Datenpaket mitgeschickt.
2. Logik-Verarbeitung im Backend (Node.js): Unser Webserver nimmt die Anfrage (z.B. auf Port 3000) entgegen. Bevor er irgendwas blind speichert, prüft unser Controller erst einmal die Berechtigungen *"Ist dieser Typ überhaupt eingeloggt und darf er das in diese Liste packen?"*. Wenn alles grün ist, bereitet das Node.js-Backend eine sichere SQL-Anweisung vor.
3. Datenbankschicht (PostgreSQL): Jetzt schickt Node.js diese parametrisierte SQL-Anfrage über eine TCP-Verbindung rüber in den PostgreSQL-Container. Die Datenbank führt die Transaktion aus, speichert das Ganze auf der Festplatte und gibt eine Bestätigung an das Backend zurück.

4. Rückmeldung an den Client: Das Backend freut sich über den Erfolg, erzeugt eine HTTP 200 OK Antwort (wieder als JSON-Paket) und schickt sie zurück. Unser JavaScript im Browser empfängt diese Antwort und aktualisiert nun dynamisch unsere HTML-Seite. Dabei sieht der Nutzer sofort, dass das Produkt auf seiner Liste gelandet ist.

4. Geforderte Dokumentation, Visualisierung und Präsentation

Für die finale Abgabe wird außerdem das benötigt:

- **Architekturbild:** Grafische Darstellung der Komponenten, Datenflüsse und Netzwerke.
- **Entity-Relationship-Model (ERM):** Modellierung des Datenbank-Schemas (Tabellen, Attribute, Kardinalitäten).
- **Mockup:** UI/UX-Planung der Webseiten-Struktur.
- **Benutzerhandbuch:** Anleitung für Kunden und Administratoren.
- **Abschlusspräsentation:** Eine 20- bis 30-minütige Präsentation vor dem Dozenten, welche die Lösung vorstellt, eine kurze Live-Demo beinhaltet und ein Fazit ("Retro und Lessons Learned") zieht.

Zusatz: Wo werden überall Docker Container benötigt?

- **Container 1:** Node.js-Backend
- **Container 2:** PostgreSQL-Datenbank
- **Optional Container 3:** Frontend, da es hier darauf ankommt, ob wir unser Frontend komplett separat bauen (mit einem eigenen Webserver).